Mandate: Frontend Design

> For the Frontend Developer. It describes **what each screen shows, which endpoints feed it, and how it behaves**, matched to the backend in `docs/API_CONTRACT.md`. Framework is your choice (React + Vite or Next.js). Everything here works with both. **Visual design is yours:** colors, typography, icons, animations and UI or chart libraries are not specified here. The wireframes show *what goes where* and can be rearranged if you have a better layout.

---

1. Design principles

1. **Every action explains itself.** Anything the agent did or wants to do shows *what*, *why* (the agent's reason) and *which rule* allowed, paused or blocked it. Judges score this as trust and explainability.
2. **Make "AI suggests, rules decide" visible.** AI content (chat answers, the agent's reasons) must be visually distinct from rule decisions (the policy engine's verdicts), for example with different labels such as "*Agent*" vs. "*Your rules*".
3. **Money first.** Big, clear numbers. Never show raw cents.
4. **Works on a phone.** Judges may open the link on their phone. Everything must work at 375 px wide.
5. **Never a blank screen.** Every list has a loading state, an empty state and an error state.

---

2. Values to display

Statuses and labels

Each of these needs a distinct, readable label (styling is your call):

Field	Values → meaning
Tier	`L0` Observe · `L1` Recommend · `L2` Ask first · `L3` Automatic
Action status	`executed` done · `pending_approval` waiting for you · `denied` blocked by your rules · `rejected` declined by you
Wish status	`on_track` · `tight` · `not_realistic` · `achieved` · `paused`
Source	`agent` · `user` · `untrusted` (from a document: must stand out)

Money and dates

- Money helper used **everywhere** (the API sends integer cents):

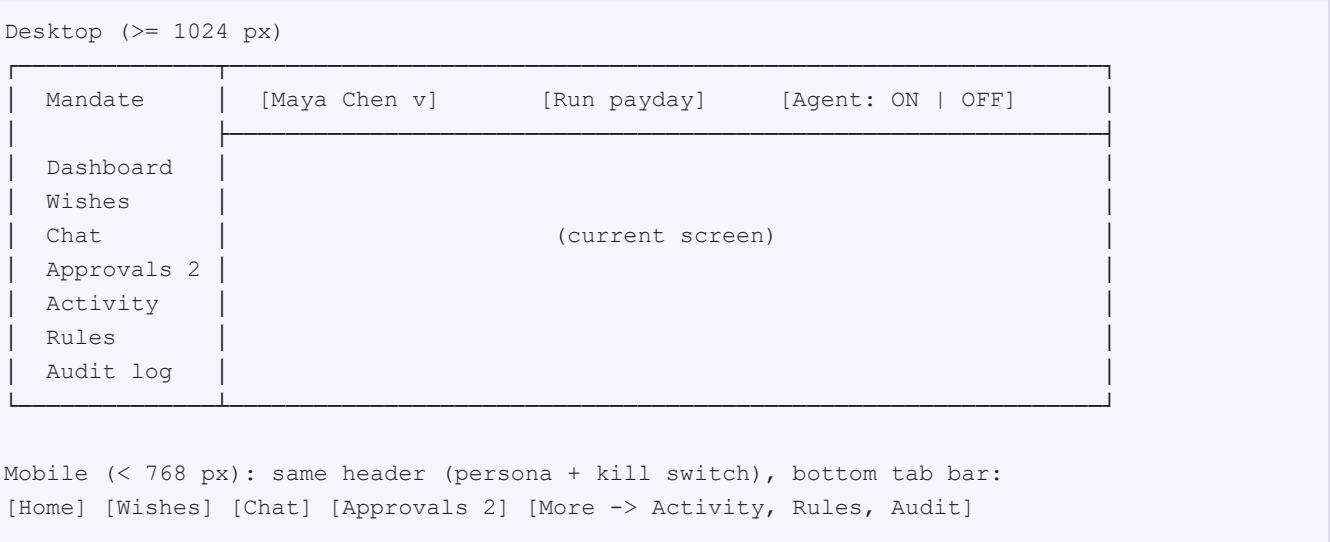
```
export const money = (cents: number, currency = "USD") =>
  (cents / 100).toLocaleString("en-US", { style: "currency", currency });
// money(18000) → "$180.00"
```

- Use tabular (fixed-width) digits for amounts so columns line up.

- Dates: short dates in lists ("Fri, Oct 16"), relative for due dates and countdowns ("in 3 days").

---

3. App shell & navigation



Header element	Behavior	Endpoint
Signed-in persona / switch demo account	Shows who's signed in. Switching = sign out + sign in as another persona (one click, using the demo accounts list); every screen reloads	`auth/me`, `auth/logout`, `auth/login`
Run payday	Demo button: triggers the agent's payday routine, then opens Activity	`POST /personas/:id/agent/run`
Agent switch (kill switch)	ON = agent may act within the rules. OFF = observe only (L0). Turning it OFF is instant; turning it ON asks for the PIN (§4.0 PIN prompt)	`GET` / `POST /personas/:id/killswitch`
Approvals badge	Count of pending approvals; updates live	`approvals` + `timeline` SSE
Injection warning banner	Full-width, prominent banner under the header when a document triggered a block; dismissible	from `documents` response / `action` events with `source: "untrusted"`

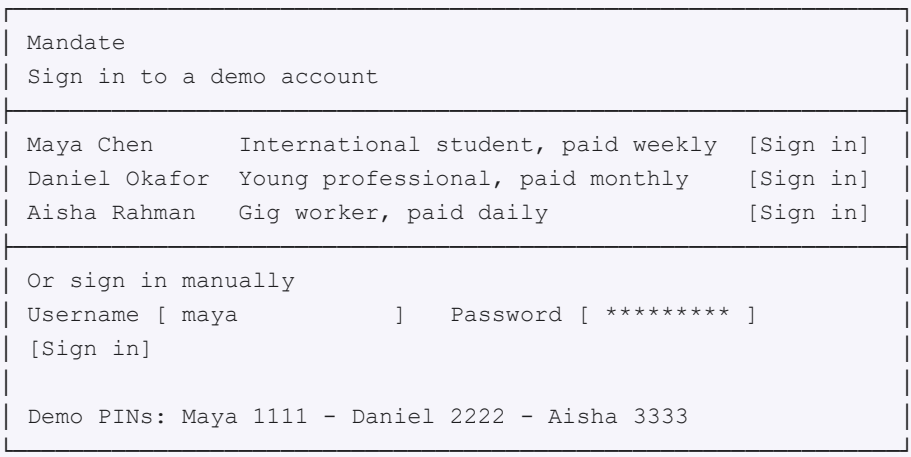
---

4. Screens

Each screen lists: purpose · layout · data · states · interactions. UI wording comes from `ui_text.csv` (written by the Story & Delivery Lead).

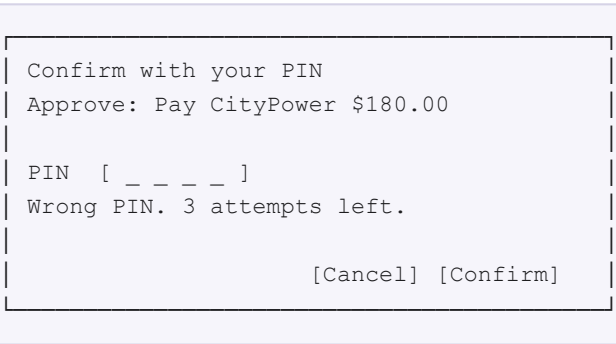
4.0 Sign-in and PIN prompt

Sign-in is the first screen when `GET /auth/me` returns `401`. The demo credentials are public on purpose, so judges can get in with one click.



- **Data:** `GET /auth/demo-accounts` for the list (it includes username, password and PIN); `POST /auth/login`; then load the app.
- **States:** wrong credentials → show the error and `attemptsLeft`; `429` → "Too many attempts. Try again in N minutes" using `retryAfterSeconds`.

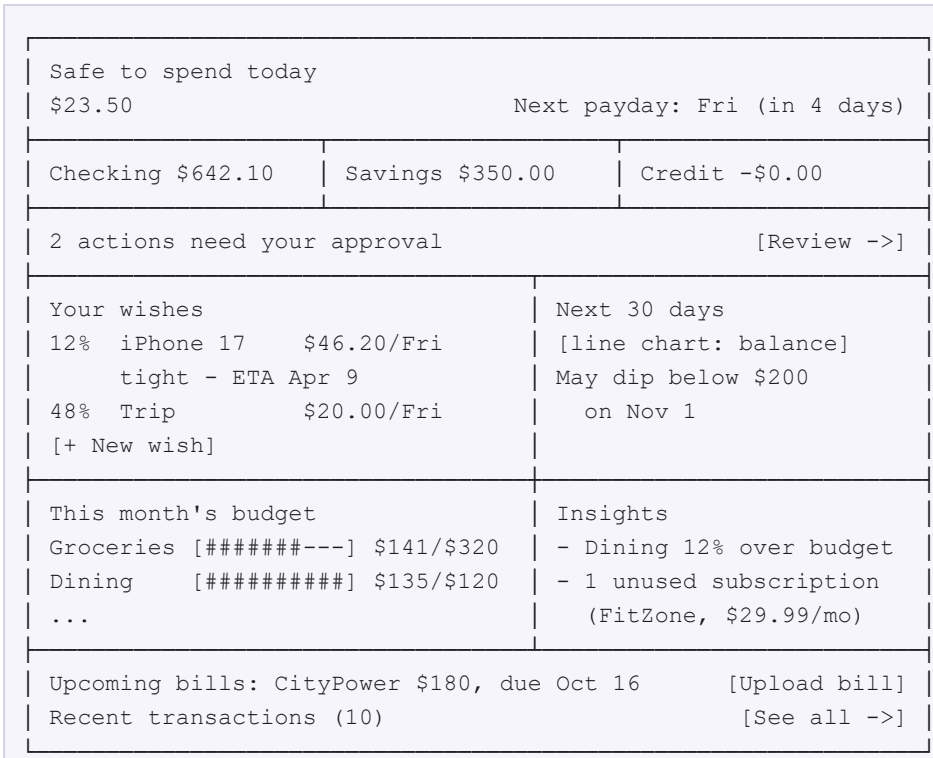
PIN prompt appears before the three risky actions: **approve** an action (§4.6), **sign** rules (§4.8), turn the agent **on** (header). Declining or turning the agent off never asks for a PIN.



- Shows *what* is being confirmed (the action summary or "Sign your rules").
- Sends the PIN with the request (`pin` field, see the contract). Wrong PIN → `403` with `attemptsLeft`; locked → `429` with `retryAfterSeconds`.
- Never store the PIN; clear the field after each attempt.

4.1 Dashboard (home)

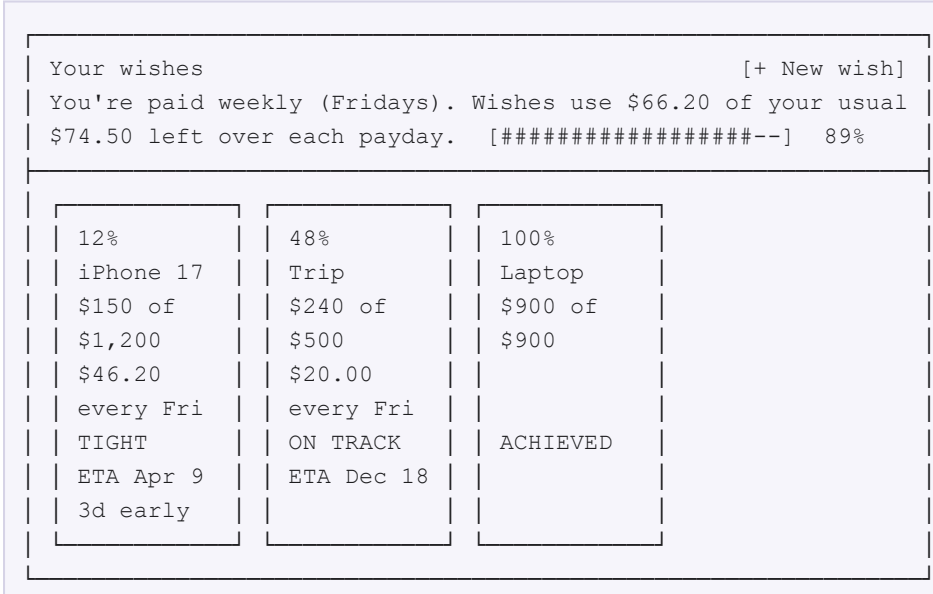
Purpose: "How am I doing, and what needs me?" in five seconds.



- **Data:** `accounts`, `budget`, `forecast`, `wishes`, `bills`, `subscriptions`, `approvals`, `pay-cycle`, `transactions?limit=10`.
- **States:** skeleton placeholders while loading; if a single card fails, show "Couldn't load this. Retry" in that card only. Don't break the page.
- **Interactions:** wish cards → Wish detail; approvals strip → Approvals; "Upload bill" → upload dialog (§4.9).

4.2 Wishes (list)

Purpose: all wishes, how they're going, and whether they fit together.



- **Data:** `GET /personas/:id/wishes` (includes `totalPerPaydayCents`, `surplusPerPaydayCents`, `overCommitted`), `pay-cycle`.

- **The commitment bar** compares total wish savings per payday with the usual surplus. If `overCommitted` is true, flag it with: "*Your wishes need more than you usually have left. Lower a target or move a date.*"
- **Wish card:** progress (%), title, saved / target, per-payday amount phrased by cadence ("$46.20 every Friday", "$6.60 a day", "$200 every month"), status, ETA + early/late.
- **Empty state:** "What are you saving for? A phone, a car, a flat. Add your first wish and we'll make a plan around your paydays." + [New wish].

4.3 New wish / edit wish (form with live plan)

Purpose: the "wow" moment. As the user types, the plan appears instantly.

```
┌──────────────────────────────────┬──────────────────────────────────────┐
│ What's your wish?                │ Your plan                            │
│ [ iPhone 17              ]       │                                      │
│                                  │   Save $46.20 every Friday           │
│ Category                         │   for 26 weeks                       │
│ (Gadget)(Vehicle)(Property)      │                                      │
│ (Travel)(Education)(Luxury)      │   Status: TIGHT                      │
│ (Other)                          │   Uses 62% of what you usually       │
│                                  │   have left each payday              │
│ How much?     [ $1,200    ]      │                                      │
│ Already saved [ $150      ]      │   Tip: cut dining by $10 a week      │
│ By when? [ Apr 12, 2027  ]       │   to make this comfortable           │
│   quick: [3mo][6mo][1yr]         │                                      │
│          [2yr][5yr]              │   ETA: Apr 9, 2027 (3d early)        │
│                                  │                                      │
│ [x] Save automatically on        │   [progress projection chart]        │
│     payday (agent, within        │                                      │
│     your rules)                  │                                      │
├──────────────────────────────────┴──────────────────────────────────────┤
│ [Cancel]  [Save wish]                                                   │
└─────────────────────────────────────────────────────────────────────────┘
```

- **Data:** `POST /personas/:id/wishes/preview` on every change (debounce 300 ms, cancel the previous request); `POST /personas/:id/wishes` on save; `PATCH /wishes/:id` when editing.
- **Validation (client side, before calling preview):** title required; amount > 0; date in the future. Show inline errors under the field.
- **Category** values: `gadget`, `vehicle`, `property`, `travel`, `education`, `luxury`, `other`. For `property` / `vehicle`, show a hint under the amount: "*Tip: for big purchases, use your down payment as the target.*"
- **Mobile:** the plan panel moves below the form and stays visible as a sticky summary ("$46.20 every Friday · tight").

4.4 Wish detail

```
┌──────────────────────────────────────────────────────────┐
│ <- Wishes        iPhone 17                    [Edit] [More] │
│      (o) 12%   $150 saved of $1,200 - TIGHT                │
│                Target Apr 12, 2027 - ETA Apr 9 (3 days early) │
├──────────────────────────────────────────────────────────┤
│ [chart: projected savings line vs. target line, actual dots] │
├──────────────────────────────────────────────────────────┤
│ Plan: $46.20 every Friday - next: Fri Oct 16 - Auto-save: ON │
│ [Add money now]                                          │
├──────────────────────────────────────────────────────────┤
│ Advice                                                   │
│ - Cut dining by $10 a week to make this comfortable.     │
├──────────────────────────────────────────────────────────┤
│ History                                                  │
│ Oct 9   +$46.20   Agent (auto-save)   done               │
│ Oct 2   +$46.20   Agent (auto-save)   done               │
│ Sep 28  +$20.00   You                 done               │
└──────────────────────────────────────────────────────────┘
```

- **Data:** `GET /wishes/:id` (`wish`, `contributions`, `projection`); `PATCH` for the auto-save toggle and pause; `DELETE` with a confirmation dialog ("Money you've saved stays in savings"). [More] holds Pause and Delete.
- **"Add money now"** → amount dialog → `POST /wishes/:id/contribute` → returns an `Action`. Show the result with the standard action result toast (§5): it may be executed, pending approval or denied, and that's the point.
- **Milestones (stretch):** at 25 / 50 / 75 / 100% acknowledge the milestone; at 100% the status becomes `achieved`.

4.5 Chat

Purpose: talk to the agent; see it use tools and propose actions inline.

```
┌──────────────────────────────────────────────────────────┐
│                      You: Can I buy a $60 concert ticket? │
│ Agent                                                    │
│   checked: forecast, wishes                              │
│   Yes. Your safe-to-spend today is $23.50, so it would come │
│   from this week's cushion. It also delays your iPhone by │
│   9 days.                                                │
│  ┌──────────────────────────────────────────────────┐    │
│  │ Agent suggests: Pay Ticketly $60.00              │    │
│  │ Your rules: payments over $50 need approval  [L2] │    │
│  │ Delays iPhone 17 by 9 days                       │    │
│  │                          [Decline] [Approve]     │    │
│  └──────────────────────────────────────────────────┘    │
├──────────────────────────────────────────────────────────┤
│ Suggestions: (How am I doing this month?) (Plan for a car) │
│ [ Ask anything about your money...              ] [Send] │
└──────────────────────────────────────────────────────────┘
```

- **Data:** `POST /personas/:id/chat` (SSE). Events:
- `text`: append `delta` to the current agent bubble (typing effect)

- `tool`: show a small "checked: forecast" line (makes the agent's work visible)
- `action`: render an inline **Action card** (§5)
- `done`: re-enable input · `error`: show "Something went wrong. Try again" in the bubble
- Keep the conversation in component state per persona (it doesn't need saving).
- **Suggestion chips** for an empty chat: "How am I doing this month?", "Can I afford a $60 ticket?", "Plan for a car in 2 years", "Which subscriptions should I cancel?"

4.6 Approvals

Purpose: everything waiting for the user, with enough context to decide in one tap.

- List of **Action cards** (§5) for `pending_approval` actions, newest first.
- Each card shows: summary, amount, the agent's reason, **the rule that requires approval** (`reasons`), tier, source, and `wishImpact` if any.
- `source: "untrusted"` cards must stand out, with a header: *"This came from a document. Check it carefully."*
- **Approve** → PIN prompt (§4.0) → `POST /actions/:id/decision` with `pin`. **Decline** → no PIN. The card leaves the list; show a toast with the result.
- **Data:** `approvals` + the `timeline` SSE stream (new cards appear live).
- **Empty state:** "Nothing waiting for you. The agent is working within your rules."

4.7 Activity (action timeline)

Purpose: the story of everything the agent did, live.

```
Today
 10:02  done     Saved $46.20 toward iPhone 17          L3  Automatic
 10:02  done     Paid CityPower $180.00                 L3  Automatic
 10:01  waiting  Cancel FitZone ($29.99/mo)             L2  Waiting for you
 09:58  blocked  Transfer $500 to new account           Blocked
                 Untrusted document - new payee - over daily limit
```

- **Data:** `GET /personas/:id/actions` for history + `timeline` SSE for live updates (prepend new items, update status in place by `id`).
- Click an item → expands to show the full reasons and the agent's rationale.
- Filters: All · Automatic · Waiting · Blocked.

4.8 Rules (mandate editor)

Purpose: write rules in plain English, see exactly what they mean, sign them.

```
Step 1: Write your rules
┌──────────────────────────────────────────────────────────┐
│ Never let checking go below $200. Save for my wishes every    │
│ payday. Ask me before any payment over $50 or to anyone new.  │
└──────────────────────────────────────────────────────────┘
                                        [Translate into rules ->]
Step 2: Check what they mean
  - Checking never goes below $200.00.
  - Wish savings run automatically on payday (up to $80.00).   L3
  - Payments over $50.00 need your approval.                   L2
  - New payees need your approval and a 24-hour wait.          L2
  ! (warnings from the compiler, if any)
  [Show technical version v]  <- the policy JSON, read-only
                                          [Edit text] [Sign rules]
Current rules: version 3 - signed Oct 15, 09:12
```

- **Data:** `GET mandate` (current), `POST mandate/compile` (step 2: nothing is saved yet), `POST mandate/sign`.
- Compiling takes a few seconds (it uses the AI). Show "Translating your rules…" with a progress indicator and keep the text editable.
- **Sign** → PIN prompt (§4.0) → `POST mandate/sign` with `pin`. Afterwards: "The agent will follow these rules from now on."
- "Show technical version" reveals the JSON. Judges like seeing that the rules are real data.

4.9 Upload bill (dialog)

- Opened from the Dashboard's bills card. Drag-and-drop or file picker; PDF only, 5 MB max.
- `POST /personas/:id/documents` (multipart, field `file`). Show a progress state ("Reading your bill…").
- **Result:** extracted fields (payee, amount, due date, account number) in a card marked `untrusted · from document`.
- If `warnings` isn't empty: a **prominent warning** inside the dialog *and* the global banner: "*A document tried to make a payment. We blocked it and nothing moved.*" This is the injection demo moment, so make it unmistakable.

4.10 Audit log

Purpose: prove nothing was hidden or changed.

```
┌──────────────────────────────────────────────────────────┐
│ Audit log             Chain status: Verified (312 entries)    │
│                       [Verify again]                          │
├──────────────────────────────────────────────────────────┤
│ #42  10:02  action_executed  Saved $46.20 toward iPhone 17    │
│      hash 9f1c...e2   prev 47aa...b1                          │
│ #41  09:58  action_denied    Blocked transfer of $500...      │
├──────────────────────────────────────────────────────────┤
│ On-chain anchors                                              │
│ Oct 17 - root 0x5be1... - base-sepolia - [View transaction ->]│
└──────────────────────────────────────────────────────────┘
```

- **Data:** `audit?personaId=…`, `audit/verify`, `anchors`.

- Hashes shortened to `first4…last2` with copy-on-click. "View transaction" opens `explorerUrl` in a new tab.
- If `valid` is false: show "Chain broken at entry #N" as a clear failure state (we'll show this on purpose in the demo by tampering in a test).

---

5. Shared components

Component	Used in	Notes
`Money`	everywhere	Wraps the `money()` helper
Tier / status / source / wish-status labels	cards, lists	Values in §2
`ActionCard`	Chat, Approvals, Activity	The most important component: summary, amount, agent reason (labelled as coming from the agent), rule reasons (labelled as coming from your rules), labels, wish impact, Approve/Decline when pending
Action result toast	after approve / contribute / payday	Three outcomes: "Done: saved $20 toward iPhone 17" · "Needs your approval" · "Blocked: over your daily limit"
`WishCard` + progress indicator	Dashboard, Wishes	Progress from `progressPct`
`CadenceText`	Wishes	`weekly + 4620` → "$46.20 every Friday"; `daily` → "a day"; `monthly` → "every month"
`StreamingText`	Chat	Appends SSE deltas
`EmptyState`, `ErrorState`, `Skeleton`	all lists	Never a blank screen
`AuthGate`	app start	Calls `GET /auth/me`; shows Sign-in on `401`, the app otherwise
`PinPrompt`	Approvals, Rules, header	§4.0
`WakingUp`	app start	Friendly full-screen state if `/api/health` takes > 3 s: "Waking up the demo server… this can take up to a minute." (hosted on Render's free plan)

---

6. Data layer

- **One API client** (`lib/api.ts`) with typed functions (`getAccounts(personaId)`, `createWish(...)`, …) using types from `@mandate/shared`. Screens never call `fetch` directly.
- **Mock mode:** an env flag (`VITE_USE_MOCKS=true` or `NEXT_PUBLIC_USE_MOCKS=true`) makes the API client return fixtures copied from the contract's examples. Build every screen in mock mode first, then flip the flag as endpoints go live. In mock mode, pretend Maya is signed in and accept any 4-digit PIN.
- **Caching:** use a data-fetching/caching approach of your choice. Include the persona in cache keys (e.g. `["wishes", personaId]`). After any action (approve, contribute, payday), refresh `accounts`, `wishes`, `approvals`, `actions`, `budget`.
- **SSE:** `chat` is a POST, so use `fetch` + `response.body.getReader()`, not `EventSource`. `timeline` is a GET, so `EventSource` works. Reconnect the timeline stream automatically if it drops.

- **Signed-in persona** comes from `GET /auth/me` and lives in a top-level context (the session itself is an `HttpOnly` cookie the browser handles; the frontend never sees or stores it).
- **On any `401`** (e.g. session expired after 8 hours): go to Sign-in. **On `403`** without `attemptsLeft`: "That isn't your account" (shouldn't happen in normal use).
- **Errors:** the API always returns `{ error }` on failure; show it in the screen's error state.

---

7. Suggested folder structure

```
apps/web/src/
├── lib/            api.ts, money.ts, sse.ts, mocks/ (fixtures from the contract)
├── components/     ActionCard, WishCard, labels, EmptyState, …
├── features/
│   ├── dashboard/
│   ├── wishes/     WishesPage, WishForm, WishDetail
│   ├── chat/
│   ├── approvals/
│   ├── activity/
│   ├── rules/
│   ├── documents/
│   └── audit/
└── app shell       layout, nav, header (persona switcher, kill switch, run payday)
```

---

8. Build order (matches the roadmap)

Date	Build	Data
Oct 13	Framework decided; app shell, nav, header, persona switcher, `money()`, API client + mock mode	personas (live), rest mock
Oct 14	Dashboard; Wishes list; New wish form with live plan	accounts/transactions live, rest mock
Oct 15	**Sign-in + PIN prompt + AuthGate**; Wish detail; ActionCard; Approvals; Activity; Chat UI (mock stream)	wishes + analytics live
Oct 16	Chat + Approvals + Activity on the real API (SSE); Rules editor; wish impact on cards	agent live
Oct 17	Upload bill + warning banners; Audit log + anchors; kill switch; Run payday; WakingUp screen	all live
Oct 18	**Freeze.** Empty/error states, mobile pass, polish	
Oct 19	Screenshots for the deck; help record the video	

---

9. Demo-critical moments (must look great)

1. **Wish form:** typing "iPhone, $1,200, 6 months" and watching the plan appear: "*$46.20 every Friday · tight*".

2. **Payday:** pressing *Run payday* and watching Activity fill with automatic actions, one approval (confirmed with the PIN) and a wish contribution.
3. **Wish impact:** "This delays your iPhone by 9 days" on an approval card and in chat.
4. **Injection:** uploading the malicious bill → the warning banner and a **Blocked** item in Activity.
5. **Audit:** "Verified" plus a working "View transaction" link.

Rehearse these with the Story & Delivery Lead using the demo script (§16 of `PRODUCT.md`).

---

10. Accessibility & quality checklist

- ☐ Works at 375 px wide; no horizontal scrolling
- ☐ Every button has a text label (or `aria-label` for icon-only buttons)
- ☐ Status is never shown by color alone: labels also have text
- ☐ Focus is visible when using the keyboard; dialogs trap focus and close with Esc
- ☐ Loading, empty and error states on every list
- ☐ Money always formatted with `money()`; dates in one consistent format